(.venv) (base) Sergio@SERHIIs-MacBook-Pro homework_buggy % pytest -v
============================= test session starts ==============================
platform darwin -- Python 3.9.12, pytest-8.4.2, pluggy-1.6.0 -- /Users/Sergio/Documents/QA_Embedded/Gate/Task5/homework_buggy/.venv/bin/python
cachedir: .pytest_cache
metadata: {'Python': '3.9.12', 'Platform': 'macOS-10.16-x86_64-i386-64bit', 'Packages': {'pytest': '8.4.2', 'pluggy': '1.6.0'}, 'Plugins': {'metadata': '3.1.1', 'html': '4.2.0'}, 'JAVA_HOME': '/Library/Java/JavaVirtualMachines/jdk-21.jdk/Contents/Home'}
rootdir: /Users/Sergio/Documents/QA_Embedded/Gate/Task5_rework/embedded_qa_wokwi_lab/homework_buggy
plugins: metadata-3.1.1, html-4.2.0
collected 49 items

test/test_resilience.py::test_boot_active_and_reset_reason PASSED [2%]
test/test_resilience.py::test_wake_time_within_tolerance[3] FAILED [4%]
test/test_resilience.py::test_wake_time_within_tolerance[5] FAILED [6%]
test/test_resilience.py::test_wake_time_within_tolerance[10] FAILED [8%]
test/test_resilience.py::test_wake_time_within_tolerance[20] FAILED [10%]
test/test_resilience.py::test_reset_reason_after_wake PASSED [12%]
test/test_resilience.py::test_operable_after_wake PASSED [14%]
test/test_resilience.py::test_repeated_sleep_cycles_and_heap FAILED [16%]
test/test_resilience.py::test_data_survives_sleep SKIPPED (firmware ...) [18%]
test/test_resilience.py::test_sensor_ok_after_wake PASSED [20%]
test/test_resilience.py::test_wdt_fires_in_time FAILED [22%]
test/test_resilience.py::test_actuator_off_after_wdt ERROR [24%]
test/test_resilience.py::test_no_boot_loop_after_wdt PASSED [26%]
test/test_resilience.py::test_powerloss_recovers PASSED [28%]
test/test_resilience.py::test_state_old_or_new_never_mixed SKIPPED (...) [30%]
test/test_resilience.py::test_data_survives_wdt SKIPPED (firmware do...) [32%]
test/test_resilience.py::test_event_counter_sane_after_recovery FAILED [34%]
test/test_resilience.py::test_data_after_mixed_cycles SKIPPED (firmw...) [36%]
test/test_resilience.py::test_invalid_sleep_args_after_recovery[-hang] FAILED [38%]
test/test_resilience.py::test_invalid_sleep_args_after_recovery[-powerloss] FAILED [40%]
test/test_resilience.py::test_invalid_sleep_args_after_recovery[-sleep] FAILED [42%]
test/test_resilience.py::test_invalid_sleep_args_after_recovery[-5-hang] FAILED [44%]
test/test_resilience.py::test_invalid_sleep_args_after_recovery[-5-powerloss]

FAILED [46%]
test/test_resilience.py::test_invalid_sleep_args_after_recovery[-5-sleep]
FAILED [48%]
test/test_resilience.py::test_invalid_sleep_args_after_recovery[0-hang] FAILED
[51%]
test/test_resilience.py::test_invalid_sleep_args_after_recovery[0-powerloss]
FAILED [53%]
test/test_resilience.py::test_invalid_sleep_args_after_recovery[0-sleep]
FAILED [55%]
test/test_resilience.py::test_invalid_sleep_args_after_recovery[abc-hang]
FAILED [57%]
test/test_resilience.py::test_invalid_sleep_args_after_recovery[abc-powerloss]
FAILED [59%]
test/test_resilience.py::test_invalid_sleep_args_after_recovery[abc-sleep]
FAILED [61%]
test/test_resilience.py::test_invalid_sleep_args_after_recovery[99999999999-
hang] FAILED [63%]
test/test_resilience.py::test_invalid_sleep_args_after_recovery[99999999999-
powerloss] FAILED [65%]
test/test_resilience.py::test_invalid_sleep_args_after_recovery[99999999999-
sleep] FAILED [67%]
test/test_resilience.py::test_invalid_sleep_args_after_recovery[3.5.1-hang]
FAILED [69%]
test/test_resilience.py::test_invalid_sleep_args_after_recovery[3.5.1-
powerloss] FAILED [71%]
test/test_resilience.py::test_invalid_sleep_args_after_recovery[3.5.1-sleep]
FAILED [73%]
test/test_resilience.py::test_garbage_after_wdt_recovery[unknown_cmd]
PASSED [75%]
test/test_resilience.py::test_garbage_after_wdt_recovery[empty_line] PASSED
[77%]
test/test_resilience.py::test_garbage_after_wdt_recovery[spaces_case]
PASSED [79%]
test/test_resilience.py::test_garbage_after_wdt_recovery[long_1kb] PASSED
[81%]
test/test_resilience.py::test_garbage_after_wdt_recovery[symbols] PASSED
[83%]
test/test_resilience.py::test_garbage_after_wdt_recovery[binary] PASSED
[85%]
test/test_resilience.py::test_command_flood FAILED [87%]
test/test_resilience.py::test_recovery_cycles[1-hang] PASSED [89%]
test/test_resilience.py::test_recovery_cycles[1-powerloss] PASSED [91%]
test/test_resilience.py::test_recovery_cycles[1-sleep] PASSED [93%]
test/test_resilience.py::test_recovery_cycles[3-hang] PASSED [95%]
test/test_resilience.py::test_recovery_cycles[3-powerloss] PASSED [97%]
test/test_resilience.py::test_recovery_cycles[3-sleep] PASSED [100%]
Serial log: logs/serial_20261014_152215.log

```
==================================== ERRORS
====================================
```
_____ ERROR at setup of test_actuator_off_after_wdt _____

```
dut = <conftest.Dut object at 0x7fa36f736ee0>
request = <SubRequest 'healthy_device' for <Function
test_actuator_off_after_wdt>>

    @pytest.fixture(autouse=True)
    def healthy_device(dut, request):
        """Each test starts with a responsive device in ACTIVE."""
        dut.flush()
>       s = dut.status()
```

conftest.py:107:

```
_ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _

self = <conftest.Dut object at 0x7fa36f736ee0>, timeout = 5.0

    def status(self, timeout=5.0):
        self.flush()
        self.send("status")
        text, t = self.read_until(r"heap", timeout)
>       assert t is not None, f"no status reply, got: {text!r}"
```
**E   AssertionError: no status reply, got: '[BOOT]
reset_reason=INT_WDT\n[WDT] enabled timeout=4000 ms\nCommands:
\r\n help\r\n status\r\n sensor\r\n relay on\r\n relay off\r\n save\r\n
sleep <seconds>\r\n hang\r\n powerloss\r\n reboot\r\n'**
E   assert None is not None

conftest.py:63: AssertionError

```
==================================== FAILURES
====================================
```
_____ test_wake_time_within_tolerance[3] _____

```
dut = <conftest.Dut object at 0x7fa36f736ee0>, n = 3

    @pytest.mark.parametrize("n", [3, 5, 10, 20])              # TC-SLP-02/03, TC-
PAR-01
    def test_wake_time_within_tolerance(dut, n):
        elapsed = do_sleep(dut, n)
        lo, hi = n * (1 - TOL), n * (1 + TOL)
>       assert lo <= elapsed <= hi, f"sleep {n}s: woke after {elapsed:.2f}s, allowed
```

{lo:.2f}-{hi:.2f}"
E AssertionError: sleep 3s: woke after 0.41s, allowed 2.55-3.45
E assert 2.55 <= 0.40943310099999985

test/test_resilience.py:61: AssertionError
_____ test_wake_time_within_tolerance[5] _____

dut = <conftest.Dut object at 0x7fa36f736ee0>, n = 5

    @pytest.mark.parametrize("n", [3, 5, 10, 20]) # TC-SLP-02/03, TC-PAR-01
    def test_wake_time_within_tolerance(dut, n):
        elapsed = do_sleep(dut, n)
        lo, hi = n * (1 - TOL), n * (1 + TOL)
>   assert lo <= elapsed <= hi, f"sleep {n}s: woke after {elapsed:.2f}s, allowed {lo:.2f}-{hi:.2f}"
E AssertionError: sleep 5s: woke after 0.59s, allowed 4.25-5.75
E assert 4.25 <= 0.5888576309999998

test/test_resilience.py:61: AssertionError
_____ test_wake_time_within_tolerance[10] _____

dut = <conftest.Dut object at 0x7fa36f736ee0>, n = 10

    @pytest.mark.parametrize("n", [3, 5, 10, 20]) # TC-SLP-02/03, TC-PAR-01
    def test_wake_time_within_tolerance(dut, n):
        elapsed = do_sleep(dut, n)
        lo, hi = n * (1 - TOL), n * (1 + TOL)
>   assert lo <= elapsed <= hi, f"sleep {n}s: woke after {elapsed:.2f}s, allowed {lo:.2f}-{hi:.2f}"
E AssertionError: sleep 10s: woke after 0.70s, allowed 8.50-11.50
E assert 8.5 <= 0.7029066220000004

test/test_resilience.py:61: AssertionError
_____ test_wake_time_within_tolerance[20] _____

dut = <conftest.Dut object at 0x7fa36f736ee0>, n = 20

    @pytest.mark.parametrize("n", [3, 5, 10, 20]) # TC-SLP-02/03, TC-PAR-01
    def test_wake_time_within_tolerance(dut, n):
        elapsed = do_sleep(dut, n)
        lo, hi = n * (1 - TOL), n * (1 + TOL)

```
>       assert lo <= elapsed <= hi, f"sleep {n}s: woke after {elapsed:.2f}s, allowed
{lo:.2f}-{hi:.2f}"
E       AssertionError: sleep 20s: woke after 0.71s, allowed 17.00-23.00
E       assert 17.0 <= 0.7057086530000003

test/test_resilience.py:61: AssertionError
_____________________________ test_repeated_sleep_cycles_and_heap
_____________________________

dut = <conftest.Dut object at 0x7fa36f736ee0>

    def test_repeated_sleep_cycles_and_heap(dut):              # TC-SLP-06
        heaps = []
        for _ in range(5):
            elapsed = do_sleep(dut, 3)
>           assert 3 * (1 - TOL) <= elapsed <= 3 * (1 + TOL)
E           assert (3 * (1 - 0.15)) <= 0.39009260499999954

test/test_resilience.py:83: AssertionError
_____________________________________ test_wdt_fires_in_time
_____________________________________

dut = <conftest.Dut object at 0x7fa36f736ee0>

    def test_wdt_fires_in_time(dut):                          # TC-WDT-01/02/03
        dut.flush()
        t0 = time.monotonic()
        dut.send("hang")
        dut.wait_boot(timeout=WDT_LIMIT + 10)
        elapsed = time.monotonic() - t0
>       assert elapsed <= WDT_LIMIT + BOOT_MARGIN, f"WDT reset after
{elapsed:.2f}s"
E       AssertionError: WDT reset after 14.59s
E       assert 14.58771402 <= (5.0 + 1.5)

test/test_resilience.py:110: AssertionError
_____________________________ test_event_counter_sane_after_recovery
_____________________________

dut = <conftest.Dut object at 0x7fa36f736ee0>

    def test_event_counter_sane_after_recovery(dut):           # TC-DAT-04
        for kind in ("sleep", "hang", "powerloss"):
            fault(dut, kind)
>           ev = dut.status().get("events", "0")

test/test_resilience.py:165:
```

```
_ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _

self = <conftest.Dut object at 0x7fa36f736ee0>, timeout = 5.0

    def status(self, timeout=5.0):
        self.flush()
        self.send("status")
        text, t = self.read_until(r"heap", timeout)
>       assert t is not None, f"no status reply, got: {text!r}"
E       AssertionError: no status reply, got: '[BOOT]
reset_reason=SOFTWARE_RESET\n[WDT] enabled timeout=4000
ms\nCommands:\r\n help\r\n status\r\n sensor\r\n relay on\r\n relay
off\r\n save\r\n sleep <seconds>\r\n hang\r\n powerloss\r\n reboot\r\n'
E       assert None is not None

conftest.py:63: AssertionError
_____ test_invalid_sleep_args_after_recovery[-hang]
_____

dut = <conftest.Dut object at 0x7fa36f736ee0>, fault_kind = 'hang', arg = ''

    @pytest.mark.parametrize("fault_kind", ["hang", "powerloss", "sleep"])
    @pytest.mark.parametrize("arg", INVALID_SLEEP)              # TC-NEG-02..06,
TC-PAR-03
    def test_invalid_sleep_args_after_recovery(dut, fault_kind, arg):
        fault(dut, fault_kind)
        dut.send(("sleep " + arg).rstrip())
        out = dut.read_for(3)
>       assert_alive_no_reboot(dut, out)

test/test_resilience.py:201:
_ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _

dut = <conftest.Dut object at 0x7fa36f736ee0>
output = '[STORE] saved\r\n[STATE] SLEEP timer=5 s\nets Jul 29 2019
12:21:46\r\n\r\nrst:0x5 (DEEPSLEEP_RESET),boot:0x13 (SPI_FA...tatus\r\n
sensor\r\n  relay on\r\n  relay off\r\n  save\r\n  sleep <seconds>\r\n  hang\r\n
powerloss\r\n  reboot\r\n'

    def assert_alive_no_reboot(dut, output):
>       assert not re.search(r"reset[_ ]reason|rst:|guru|panic|abort", output, re.I),
f"reboot/crash: {output!r}"
E       AssertionError: reboot/crash: '[STORE] saved\r\n[STATE] SLEEP
timer=5 s\nets Jul 29 2019 12:21:46\r\n\r\nrst:0x5
(DEEPSLEEP_RESET),boot:0x13 (SPI_FAST_FLASH_BOOT)\r\nconfigsip: 0,
SPIWP:0xee\r\nclk_drv:0x00,q_drv:0x00,d_drv:0x00,cs0_drv:0x00,hd_dr
v:0x00,wp_drv:0x00\r\nmode:DIO, clock
```

div:2\r\nload:0x3fff0030,len:1156\r\nload:0x40078000,len:11456\r\nho 0 tail 12 room 4\r\nload:0x40080400,len:2972\r\nentry 0x400805dc\r\n[BOOT] reset_reason=DEEP_SLEEP\n[WAKE] cause=4\n[WDT] enabled timeout=4000 ms\nCommands:\r\n help\r\n status\r\n sensor\r\n relay on\r\n relay off\r\n save\r\n sleep <seconds>\r\n hang\r\n powerloss\r\n reboot\r\n'
E   assert not <re.Match object; span=(67, 71), match='rst:'>
E   + where <re.Match object; span=(67, 71), match='rst:'> = <function search at 0x7fa36afc9f70>('reset[_]reason|rst:|guru|panic|abort', '[STORE] saved\r\n[STATE] SLEEP timer=5 s\nets Jul 29 2019 12:21:46\r\n\r\nrst:0x5 (DEEPSLEEP_RESET),boot:0x13 (SPI_FAST_FLASH_BOOT)\r\nconfigsip: 0, SPIWP:0xee\r\nclk_drv:0x00,q_drv:0x00,d_drv:0x00,cs0_drv:0x00,hd_drv:0x00,wp_drv:0x00\r\nmode:DIO, clock div:2\r\nload:0x3fff0030,len:1156\r\nload:0x40078000,len:11456\r\nho 0 tail 12 room 4\r\nload:0x40080400,len:2972\r\nentry 0x400805dc\r\n[BOOT] reset_reason=DEEP_SLEEP\n[WAKE] cause=4\n[WDT] enabled timeout=4000 ms\nCommands:\r\n help\r\n status\r\n sensor\r\n relay on\r\n relay off\r\n save\r\n sleep <seconds>\r\n hang\r\n powerloss\r\n reboot\r\n', re.IGNORECASE)
E   +   where <function search at 0x7fa36afc9f70> = re.search
E   +   and   re.IGNORECASE = re.I

test/test_resilience.py:187: AssertionError
_____ test_invalid_sleep_args_after_recovery[-powerloss] _____

dut = <conftest.Dut object at 0x7fa36f736ee0>, fault_kind = 'powerloss'
arg = ''

    @pytest.mark.parametrize("fault_kind", ["hang", "powerloss", "sleep"])
    @pytest.mark.parametrize("arg", INVALID_SLEEP)   # TC-NEG-02..06, TC-PAR-03
    def test_invalid_sleep_args_after_recovery(dut, fault_kind, arg):
        fault(dut, fault_kind)
        dut.send(("sleep " + arg).rstrip())
        out = dut.read_for(3)
>   assert_alive_no_reboot(dut, out)

test/test_resilience.py:201:
_ _

dut = <conftest.Dut object at 0x7fa36f736ee0>
output = '[STORE] saved\r\n[STATE] SLEEP timer=5 s\nets Jul 29 2019 12:21:46\r\n\r\nrst:0x5 (DEEPSLEEP_RESET),boot:0x13 (SPI_FA...tatus\r\n sensor\r\n relay on\r\n relay off\r\n save\r\n sleep <seconds>\r\n hang\r\n powerloss\r\n reboot\r\n'

```
    def assert_alive_no_reboot(dut, output):
>       assert not re.search(r"reset[_ ]reason|rst:|guru|panic|abort", output, re.I),
f"reboot/crash: {output!r}"
E       AssertionError: reboot/crash: '[STORE] saved\r\n[STATE] SLEEP
timer=5 s\nets Jul 29 2019 12:21:46\r\n\r\nrst:0x5
(DEEPSLEEP_RESET),boot:0x13 (SPI_FAST_FLASH_BOOT)\r\nconfigsip: 0,
SPIWP:0xee\r\nclk_drv:0x00,q_drv:0x00,d_drv:0x00,cs0_drv:0x00,hd_dr
v:0x00,wp_drv:0x00\r\nmode:DIO, clock
div:2\r\nload:0x3fff0030,len:1156\r\nload:0x40078000,len:11456\r\nho 0
tail 12 room 4\r\nload:0x40080400,len:2972\r\nentry
0x400805dc\r\n[BOOT] reset_reason=DEEP_SLEEP\n[WAKE]
cause=4\n[WDT] enabled timeout=4000 ms\nCommands:\r\n  help\r\n
status\r\n  sensor\r\n  relay on\r\n  relay off\r\n  save\r\n  sleep
<seconds>\r\n  hang\r\n  powerloss\r\n  reboot\r\n'
E       assert not <re.Match object; span=(67, 71), match='rst:'>
E        +  where <re.Match object; span=(67, 71), match='rst:'> = <function
search at 0x7fa36afc9f70>('reset[_ ]reason|rst:|guru|panic|abort',
'[STORE] saved\r\n[STATE] SLEEP timer=5 s\nets Jul 29 2019
12:21:46\r\n\r\nrst:0x5 (DEEPSLEEP_RESET),boot:0x13
(SPI_FAST_FLASH_BOOT)\r\nconfigsip: 0,
SPIWP:0xee\r\nclk_drv:0x00,q_drv:0x00,d_drv:0x00,cs0_drv:0x00,hd_dr
v:0x00,wp_drv:0x00\r\nmode:DIO, clock
div:2\r\nload:0x3fff0030,len:1156\r\nload:0x40078000,len:11456\r\nho 0
tail 12 room 4\r\nload:0x40080400,len:2972\r\nentry
0x400805dc\r\n[BOOT] reset_reason=DEEP_SLEEP\n[WAKE]
cause=4\n[WDT] enabled timeout=4000 ms\nCommands:\r\n  help\r\n
status\r\n  sensor\r\n  relay on\r\n  relay off\r\n  save\r\n  sleep
<seconds>\r\n  hang\r\n  powerloss\r\n  reboot\r\n', re.IGNORECASE)
E       +    where <function search at 0x7fa36afc9f70> = re.search
E       +    and   re.IGNORECASE = re.I

test/test_resilience.py:187: AssertionError
_____________________ test_invalid_sleep_args_after_recovery[-sleep]
_____________________

dut = <conftest.Dut object at 0x7fa36f736ee0>, fault_kind = 'sleep', arg = ''

    @pytest.mark.parametrize("fault_kind", ["hang", "powerloss", "sleep"])
    @pytest.mark.parametrize("arg", INVALID_SLEEP)              # TC-NEG-02..06,
TC-PAR-03
    def test_invalid_sleep_args_after_recovery(dut, fault_kind, arg):
        fault(dut, fault_kind)
        dut.send(("sleep " + arg).rstrip())
        out = dut.read_for(3)
>       assert_alive_no_reboot(dut, out)
```

test/test_resilience.py:201:
_ _

dut = <conftest.Dut object at 0x7fa36f736ee0>
output = '[STORE] saved\r\n[STATE] SLEEP timer=5 s\nets Jul 29 2019 12:21:46\r\n\r\nrst:0x5 (DEEPSLEEP_RESET),boot:0x13 (SPI_FA...tatus\r\n sensor\r\n relay on\r\n relay off\r\n save\r\n sleep <seconds>\r\n hang\r\n powerloss\r\n reboot\r\n'

    def assert_alive_no_reboot(dut, output):
>   assert not re.search(r"reset[_]reason|rst:|guru|panic|abort", output, re.I), f"reboot/crash: {output!r}"
E   AssertionError: reboot/crash: '[STORE] saved\r\n[STATE] SLEEP timer=5 s\nets Jul 29 2019 12:21:46\r\n\r\nrst:0x5 (DEEPSLEEP_RESET),boot:0x13 (SPI_FAST_FLASH_BOOT)\r\nconfigsip: 0, SPIWP:0xee\r\nclk_drv:0x00,q_drv:0x00,d_drv:0x00,cs0_drv:0x00,hd_drv:0x00,wp_drv:0x00\r\nmode:DIO, clock div:2\r\nload:0x3fff0030,len:1156\r\nload:0x40078000,len:11456\r\nho 0 tail 12 room 4\r\nload:0x40080400,len:2972\r\nentry 0x400805dc\r\n[BOOT] reset_reason=DEEP_SLEEP\n[WAKE] cause=4\n[WDT] enabled timeout=4000 ms\nCommands:\r\n help\r\n status\r\n sensor\r\n relay on\r\n relay off\r\n save\r\n sleep <seconds>\r\n hang\r\n powerloss\r\n reboot\r\n'
E   assert not <re.Match object; span=(67, 71), match='rst:'>
E   + where <re.Match object; span=(67, 71), match='rst:'> = <function search at 0x7fa36afc9f70>('reset[_]reason|rst:|guru|panic|abort', '[STORE] saved\r\n[STATE] SLEEP timer=5 s\nets Jul 29 2019 12:21:46\r\n\r\nrst:0x5 (DEEPSLEEP_RESET),boot:0x13 (SPI_FAST_FLASH_BOOT)\r\nconfigsip: 0, SPIWP:0xee\r\nclk_drv:0x00,q_drv:0x00,d_drv:0x00,cs0_drv:0x00,hd_drv:0x00,wp_drv:0x00\r\nmode:DIO, clock div:2\r\nload:0x3fff0030,len:1156\r\nload:0x40078000,len:11456\r\nho 0 tail 12 room 4\r\nload:0x40080400,len:2972\r\nentry 0x400805dc\r\n[BOOT] reset_reason=DEEP_SLEEP\n[WAKE] cause=4\n[WDT] enabled timeout=4000 ms\nCommands:\r\n help\r\n status\r\n sensor\r\n relay on\r\n relay off\r\n save\r\n sleep <seconds>\r\n hang\r\n powerloss\r\n reboot\r\n', re.IGNORECASE)
E   +   where <function search at 0x7fa36afc9f70> = re.search
E   +   and   re.IGNORECASE = re.I

test/test_resilience.py:187: AssertionError
_____ test_invalid_sleep_args_after_recovery[-5-hang] _____

dut = <conftest.Dut object at 0x7fa36f736ee0>, fault_kind = 'hang', arg = '-5'

```
    @pytest.mark.parametrize("fault_kind", ["hang", "powerloss", "sleep"])
    @pytest.mark.parametrize("arg", INVALID_SLEEP)              # TC-NEG-02..06,
TC-PAR-03
    def test_invalid_sleep_args_after_recovery(dut, fault_kind, arg):
        fault(dut, fault_kind)
        dut.send(("sleep " + arg).rstrip())
        out = dut.read_for(3)
>       assert_alive_no_reboot(dut, out)
```

test/test_resilience.py:201:

```
_ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _
```

```
dut = <conftest.Dut object at 0x7fa36f736ee0>
output = '[STORE] saved\r\n[STATE] SLEEP timer=60 s\nets Jul 29 2019
12:21:46\r\n\r\nrst:0x8 (TG1WDT_SYS_RESET),boot:0x13 (SPI_...tatus\r\n
sensor\r\n  relay on\r\n  relay off\r\n  save\r\n  sleep <seconds>\r\n  hang\r\n
powerloss\r\n  reboot\r\n'

    def assert_alive_no_reboot(dut, output):
>       assert not re.search(r"reset[_ ]reason|rst:|guru|panic|abort", output, re.I),
f"reboot/crash: {output!r}"
E       AssertionError: reboot/crash: '[STORE] saved\r\n[STATE] SLEEP
timer=60 s\nets Jul 29 2019 12:21:46\r\n\r\nrst:0x8
(TG1WDT_SYS_RESET),boot:0x13 (SPI_FAST_FLASH_BOOT)\r\nconfigsip:
0,
SPIWP:0xee\r\nclk_drv:0x00,q_drv:0x00,d_drv:0x00,cs0_drv:0x00,hd_dr
v:0x00,wp_drv:0x00\r\nmode:DIO, clock
div:2\r\nload:0x3fff0030,len:1156\r\nload:0x40078000,len:11456\r\nho 0
tail 12 room 4\r\nload:0x40080400,len:2972\r\nentry
0x400805dc\r\n[BOOT] reset_reason=INT_WDT\n[WDT] enabled
timeout=4000 ms\nCommands:\r\n  help\r\n  status\r\n  sensor\r\n  relay
on\r\n  relay off\r\n  save\r\n  sleep <seconds>\r\n  hang\r\n  powerloss\r\n
reboot\r\n'
E       assert not <re.Match object; span=(68, 72), match='rst:'>
E        +  where <re.Match object; span=(68, 72), match='rst:'> = <function
search at 0x7fa36afc9f70>('reset[_ ]reason|rst:|guru|panic|abort',
'[STORE] saved\r\n[STATE] SLEEP timer=60 s\nets Jul 29 2019
12:21:46\r\n\r\nrst:0x8 (TG1WDT_SYS_RESET),boot:0x13
(SPI_FAST_FLASH_BOOT)\r\nconfigsip: 0,
SPIWP:0xee\r\nclk_drv:0x00,q_drv:0x00,d_drv:0x00,cs0_drv:0x00,hd_dr
v:0x00,wp_drv:0x00\r\nmode:DIO, clock
div:2\r\nload:0x3fff0030,len:1156\r\nload:0x40078000,len:11456\r\nho 0
tail 12 room 4\r\nload:0x40080400,len:2972\r\nentry
0x400805dc\r\n[BOOT] reset_reason=INT_WDT\n[WDT] enabled
timeout=4000 ms\nCommands:\r\n  help\r\n  status\r\n  sensor\r\n  relay
on\r\n  relay off\r\n  save\r\n  sleep <seconds>\r\n  hang\r\n  powerloss\r\n
```

reboot\r\n', re.IGNORECASE)
E + where <function search at 0x7fa36afc9f70> = re.search
E + and re.IGNORECASE = re.I

test/test_resilience.py:187: AssertionError
_____ test_invalid_sleep_args_after_recovery[-5-powerloss] _____

dut = <conftest.Dut object at 0x7fa36f736ee0>, fault_kind = 'powerloss'
arg = '-5'

```
    @pytest.mark.parametrize("fault_kind", ["hang", "powerloss", "sleep"])
    @pytest.mark.parametrize("arg", INVALID_SLEEP)              # TC-NEG-02..06, TC-PAR-03
    def test_invalid_sleep_args_after_recovery(dut, fault_kind, arg):
        fault(dut, fault_kind)
        dut.send(("sleep " + arg).rstrip())
        out = dut.read_for(3)
>       assert_alive_no_reboot(dut, out)
```

test/test_resilience.py:201:
_ _

dut = <conftest.Dut object at 0x7fa36f736ee0>
output = 'reason=SOFTWARE_RESET\n[WDT] enabled timeout=4000 ms\nCommands:\r\n help\r\n status\r\n sensor\r\n relay on\r\n ...tatus\r\n sensor\r\n relay on\r\n relay off\r\n save\r\n sleep <seconds>\r\n hang\r\n powerloss\r\n reboot\r\n'

```
    def assert_alive_no_reboot(dut, output):
>       assert not re.search(r"reset[_ ]reason|rst:|guru|panic|abort", output, re.I), f"reboot/crash: {output!r}"
```
E AssertionError: reboot/crash: 'reason=SOFTWARE_RESET\n[WDT] enabled timeout=4000 ms\nCommands:\r\n help\r\n status\r\n sensor\r\n relay on\r\n relay off\r\n save\r\n sleep <seconds>\r\n hang\r\n powerloss\r\n reboot\r\n[STORE] saved\r\n[STATE] SLEEP timer=60 s\nets Jul 29 2019 12:21:46\r\n\r\nrst:0x8 (TG1WDT_SYS_RESET),boot:0x13 (SPI_FAST_FLASH_BOOT)\r\nconfigsip: 0, SPIWP:0xee\r\nclk_drv:0x00,q_drv:0x00,d_drv:0x00,cs0_drv:0x00,hd_drv:0x00,wp_drv:0x00\r\nmode:DIO, clock div:2\r\nload:0x3fff0030,len:1156\r\nload:0x40078000,len:11456\r\nho 0 tail 12 room 4\r\nload:0x40080400,len:2972\r\nentry 0x400805dc\r\n[BOOT] reset_reason=INT_WDT\n[WDT] enabled timeout=4000 ms\nCommands:\r\n help\r\n status\r\n sensor\r\n relay on\r\n relay off\r\n save\r\n sleep <seconds>\r\n hang\r\n powerloss\r\n reboot\r\n'
E assert not <re.Match object; span=(242, 246), match='rst:'>

E + where <re.Match object; span=(242, 246), match='rst:'> = <function search at 0x7fa36afc9f70>('reset[_]reason|rst:|guru|panic|abort', 'reason=SOFTWARE_RESET\n[WDT] enabled timeout=4000 ms\nCommands:\r\n help\r\n status\r\n sensor\r\n relay on\r\n relay off\r\n save\r\n sleep <seconds>\r\n hang\r\n powerloss\r\n reboot\r\n[STORE] saved\r\n[STATE] SLEEP timer=60 s\nets Jul 29 2019 12:21:46\r\n\r\nrst:0x8 (TG1WDT_SYS_RESET),boot:0x13 (SPI_FAST_FLASH_BOOT)\r\nconfigsip: 0, SPIWP:0xee\r\nclk_drv:0x00,q_drv:0x00,d_drv:0x00,cs0_drv:0x00,hd_drv:0x00,wp_drv:0x00\r\nmode:DIO, clock div:2\r\nload:0x3fff0030,len:1156\r\nload:0x40078000,len:11456\r\nho 0 tail 12 room 4\r\nload:0x40080400,len:2972\r\nentry 0x400805dc\r\n[BOOT] reset_reason=INT_WDT\n[WDT] enabled timeout=4000 ms\nCommands:\r\n help\r\n status\r\n sensor\r\n relay on\r\n relay off\r\n save\r\n sleep <seconds>\r\n hang\r\n powerloss\r\n reboot\r\n', re.IGNORECASE)
E + where <function search at 0x7fa36afc9f70> = re.search
E + and re.IGNORECASE = re.I

test/test_resilience.py:187: AssertionError
_____ test_invalid_sleep_args_after_recovery[-5-sleep] _____

dut = <conftest.Dut object at 0x7fa36f736ee0>, fault_kind = 'sleep', arg = '-5'

```
    @pytest.mark.parametrize("fault_kind", ["hang", "powerloss", "sleep"])
    @pytest.mark.parametrize("arg", INVALID_SLEEP)              # TC-NEG-02..06, TC-PAR-03
    def test_invalid_sleep_args_after_recovery(dut, fault_kind, arg):
        fault(dut, fault_kind)
        dut.send(("sleep " + arg).rstrip())
        out = dut.read_for(3)
>       assert_alive_no_reboot(dut, out)
```

test/test_resilience.py:201:
_ _

dut = <conftest.Dut object at 0x7fa36f736ee0>
output = '[STORE] saved\r\n[STATE] SLEEP timer=60 s\nets Jul 29 2019 12:21:46\r\n\r\nrst:0x8 (TG1WDT_SYS_RESET),boot:0x13 (SPI_...tatus\r\n sensor\r\n relay on\r\n relay off\r\n save\r\n sleep <seconds>\r\n hang\r\n powerloss\r\n reboot\r\n'

```
    def assert_alive_no_reboot(dut, output):
>       assert not re.search(r"reset[_ ]reason|rst:|guru|panic|abort", output, re.I), f"reboot/crash: {output!r}"
```
E AssertionError: reboot/crash: '[STORE] saved\r\n[STATE] SLEEP

timer=60 s\nets Jul 29 2019 12:21:46\r\n\r\nrst:0x8 (TG1WDT_SYS_RESET),boot:0x13 (SPI_FAST_FLASH_BOOT)\r\nconfigsip: 0, SPIWP:0xee\r\nclk_drv:0x00,q_drv:0x00,d_drv:0x00,cs0_drv:0x00,hd_drv:0x00,wp_drv:0x00\r\nmode:DIO, clock div:2\r\nload:0x3fff0030,len:1156\r\nload:0x40078000,len:11456\r\nho 0 tail 12 room 4\r\nload:0x40080400,len:2972\r\nentry 0x400805dc\r\n[BOOT] reset_reason=INT_WDT\n[WDT] enabled timeout=4000 ms\nCommands:\r\n help\r\n status\r\n sensor\r\n relay on\r\n relay off\r\n save\r\n sleep <seconds>\r\n hang\r\n powerloss\r\n reboot\r\n'
E assert not <re.Match object; span=(68, 72), match='rst:'>
E + where <re.Match object; span=(68, 72), match='rst:'> = <function search at 0x7fa36afc9f70>('reset[_]reason|rst:|guru|panic|abort', '[STORE] saved\r\n[STATE] SLEEP timer=60 s\nets Jul 29 2019 12:21:46\r\n\r\nrst:0x8 (TG1WDT_SYS_RESET),boot:0x13 (SPI_FAST_FLASH_BOOT)\r\nconfigsip: 0, SPIWP:0xee\r\nclk_drv:0x00,q_drv:0x00,d_drv:0x00,cs0_drv:0x00,hd_drv:0x00,wp_drv:0x00\r\nmode:DIO, clock div:2\r\nload:0x3fff0030,len:1156\r\nload:0x40078000,len:11456\r\nho 0 tail 12 room 4\r\nload:0x40080400,len:2972\r\nentry 0x400805dc\r\n[BOOT] reset_reason=INT_WDT\n[WDT] enabled timeout=4000 ms\nCommands:\r\n help\r\n status\r\n sensor\r\n relay on\r\n relay off\r\n save\r\n sleep <seconds>\r\n hang\r\n powerloss\r\n reboot\r\n', re.IGNORECASE)
E + where <function search at 0x7fa36afc9f70> = re.search
E + and re.IGNORECASE = re.I

test/test_resilience.py:187: AssertionError
_____ test_invalid_sleep_args_after_recovery[0-hang] _____

dut = <conftest.Dut object at 0x7fa36f736ee0>, fault_kind = 'hang', arg = '0'

```
    @pytest.mark.parametrize("fault_kind", ["hang", "powerloss", "sleep"])
    @pytest.mark.parametrize("arg", INVALID_SLEEP)              # TC-NEG-02..06, TC-PAR-03
    def test_invalid_sleep_args_after_recovery(dut, fault_kind, arg):
        fault(dut, fault_kind)
        dut.send(("sleep " + arg).rstrip())
        out = dut.read_for(3)
>       assert_alive_no_reboot(dut, out)
```

test/test_resilience.py:201:

_ _

dut = <conftest.Dut object at 0x7fa36f736ee0>

output = '[STORE] saved\r\n[STATE] SLEEP timer=1 s\nets Jul 29 2019 12:21:46\r\n\r\nrst:0x5 (DEEPSLEEP_RESET),boot:0x13 (SPI_FA...tatus\r\n sensor\r\n relay on\r\n relay off\r\n save\r\n sleep <seconds>\r\n hang\r\n powerloss\r\n reboot\r\n'

    def assert_alive_no_reboot(dut, output):
>   assert not re.search(r"reset[_]reason|rst:|guru|panic|abort", output, re.I), f"reboot/crash: {output!r}"
E   AssertionError: reboot/crash: '[STORE] saved\r\n[STATE] SLEEP timer=1 s\nets Jul 29 2019 12:21:46\r\n\r\nrst:0x5 (DEEPSLEEP_RESET),boot:0x13 (SPI_FAST_FLASH_BOOT)\r\nconfigsip: 0, SPIWP:0xee\r\nclk_drv:0x00,q_drv:0x00,d_drv:0x00,cs0_drv:0x00,hd_drv:0x00,wp_drv:0x00\r\nmode:DIO, clock div:2\r\nload:0x3fff0030,len:1156\r\nload:0x40078000,len:11456\r\nho 0 tail 12 room 4\r\nload:0x40080400,len:2972\r\nentry 0x400805dc\r\n[BOOT] reset_reason=DEEP_SLEEP\n[WAKE] cause=4\n[WDT] enabled timeout=4000 ms\nCommands:\r\n help\r\n status\r\n sensor\r\n relay on\r\n relay off\r\n save\r\n sleep <seconds>\r\n hang\r\n powerloss\r\n reboot\r\n'
E   assert not <re.Match object; span=(67, 71), match='rst:'>
E   + where <re.Match object; span=(67, 71), match='rst:'> = <function search at 0x7fa36afc9f70>('reset[_]reason|rst:|guru|panic|abort', '[STORE] saved\r\n[STATE] SLEEP timer=1 s\nets Jul 29 2019 12:21:46\r\n\r\nrst:0x5 (DEEPSLEEP_RESET),boot:0x13 (SPI_FAST_FLASH_BOOT)\r\nconfigsip: 0, SPIWP:0xee\r\nclk_drv:0x00,q_drv:0x00,d_drv:0x00,cs0_drv:0x00,hd_drv:0x00,wp_drv:0x00\r\nmode:DIO, clock div:2\r\nload:0x3fff0030,len:1156\r\nload:0x40078000,len:11456\r\nho 0 tail 12 room 4\r\nload:0x40080400,len:2972\r\nentry 0x400805dc\r\n[BOOT] reset_reason=DEEP_SLEEP\n[WAKE] cause=4\n[WDT] enabled timeout=4000 ms\nCommands:\r\n help\r\n status\r\n sensor\r\n relay on\r\n relay off\r\n save\r\n sleep <seconds>\r\n hang\r\n powerloss\r\n reboot\r\n', re.IGNORECASE)
E   +   where <function search at 0x7fa36afc9f70> = re.search
E   +   and   re.IGNORECASE = re.I

test/test_resilience.py:187: AssertionError
_____ test_invalid_sleep_args_after_recovery[0-powerloss] _____

dut = <conftest.Dut object at 0x7fa36f736ee0>, fault_kind = 'powerloss'
arg = '0'

    @pytest.mark.parametrize("fault_kind", ["hang", "powerloss", "sleep"])
    @pytest.mark.parametrize("arg", INVALID_SLEEP)   # TC-NEG-02..06, TC-PAR-03
    def test_invalid_sleep_args_after_recovery(dut, fault_kind, arg):

```
        fault(dut, fault_kind)
        dut.send(("sleep " + arg).rstrip())
        out = dut.read_for(3)
>       assert_alive_no_reboot(dut, out)
```

test/test_resilience.py:201:

_ _

```
dut = <conftest.Dut object at 0x7fa36f736ee0>
output = '[BOOT] reset_reason=SOFTWARE_RESET\n[WDT] enabled
timeout=4000 ms\nCommands:\r\n  help\r\n  status\r\n  sensor\r\n  r...tatus\r\n
sensor\r\n  relay on\r\n  relay off\r\n  save\r\n  sleep <seconds>\r\n  hang\r\n
powerloss\r\n  reboot\r\n'

    def assert_alive_no_reboot(dut, output):
>       assert not re.search(r"reset[_ ]reason|rst:|guru|panic|abort", output, re.I),
f"reboot/crash: {output!r}"
E       AssertionError: reboot/crash: '[BOOT]
reset_reason=SOFTWARE_RESET\n[WDT] enabled timeout=4000
ms\nCommands:\r\n  help\r\n  status\r\n  sensor\r\n  relay on\r\n  relay
off\r\n  save\r\n  sleep <seconds>\r\n  hang\r\n  powerloss\r\n
reboot\r\n[STORE] saved\r\n[STATE] SLEEP timer=1 s\nets Jul 29 2019
12:21:46\r\n\r\nrst:0x5 (DEEPSLEEP_RESET),boot:0x13
(SPI_FAST_FLASH_BOOT)\r\nconfigsip: 0,
SPIWP:0xee\r\nclk_drv:0x00,q_drv:0x00,d_drv:0x00,cs0_drv:0x00,hd_dr
v:0x00,wp_drv:0x00\r\nmode:DIO, clock
div:2\r\nload:0x3fff0030,len:1156\r\nload:0x40078000,len:11456\r\nho 0
tail 12 room 4\r\nload:0x40080400,len:2972\r\nentry
0x400805dc\r\n[BOOT] reset_reason=DEEP_SLEEP\n[WAKE]
cause=4\n[WDT] enabled timeout=4000 ms\nCommands:\r\n  help\r\n
status\r\n  sensor\r\n  relay on\r\n  relay off\r\n  save\r\n  sleep
<seconds>\r\n  hang\r\n  powerloss\r\n  reboot\r\n'
E       assert not <re.Match object; span=(7, 19), match='reset_reason'>
E        +  where <re.Match object; span=(7, 19), match='reset_reason'> =
<function search at 0x7fa36afc9f70>('reset[_ ]reason|rst:|guru|panic|
abort', '[BOOT] reset_reason=SOFTWARE_RESET\n[WDT] enabled
timeout=4000 ms\nCommands:\r\n  help\r\n  status\r\n  sensor\r\n  relay
on\r\n  relay off\r\n  save\r\n  sleep <seconds>\r\n  hang\r\n  powerloss\r\n
reboot\r\n[STORE] saved\r\n[STATE] SLEEP timer=1 s\nets Jul 29 2019
12:21:46\r\n\r\nrst:0x5 (DEEPSLEEP_RESET),boot:0x13
(SPI_FAST_FLASH_BOOT)\r\nconfigsip: 0,
SPIWP:0xee\r\nclk_drv:0x00,q_drv:0x00,d_drv:0x00,cs0_drv:0x00,hd_dr
v:0x00,wp_drv:0x00\r\nmode:DIO, clock
div:2\r\nload:0x3fff0030,len:1156\r\nload:0x40078000,len:11456\r\nho 0
tail 12 room 4\r\nload:0x40080400,len:2972\r\nentry
0x400805dc\r\n[BOOT] reset_reason=DEEP_SLEEP\n[WAKE]
cause=4\n[WDT] enabled timeout=4000 ms\nCommands:\r\n  help\r\n
```

status\r\n sensor\r\n relay on\r\n relay off\r\n save\r\n sleep <seconds>\r\n hang\r\n powerloss\r\n reboot\r\n', re.IGNORECASE)
E   + where <function search at 0x7fa36afc9f70> = re.search
E   + and   re.IGNORECASE = re.I

test/test_resilience.py:187: AssertionError
_____ test_invalid_sleep_args_after_recovery[0-sleep] _____

dut = <conftest.Dut object at 0x7fa36f736ee0>, fault_kind = 'sleep', arg = '0'

    @pytest.mark.parametrize("fault_kind", ["hang", "powerloss", "sleep"])
    @pytest.mark.parametrize("arg", INVALID_SLEEP)   # TC-NEG-02..06, TC-PAR-03
    def test_invalid_sleep_args_after_recovery(dut, fault_kind, arg):
        fault(dut, fault_kind)
        dut.send(("sleep " + arg).rstrip())
        out = dut.read_for(3)
>   assert_alive_no_reboot(dut, out)

test/test_resilience.py:201:
_ _

dut = <conftest.Dut object at 0x7fa36f736ee0>
output = '[STORE] saved\r\n[STATE] SLEEP timer=1 s\nets Jul 29 2019 12:21:46\r\n\r\nrst:0x5 (DEEPSLEEP_RESET),boot:0x13 (SPI_FA...tatus\r\n sensor\r\n relay on\r\n relay off\r\n save\r\n sleep <seconds>\r\n hang\r\n powerloss\r\n reboot\r\n'

    def assert_alive_no_reboot(dut, output):
>   assert not re.search(r"reset[_]reason|rst:|guru|panic|abort", output, re.I), f"reboot/crash: {output!r}"
E   AssertionError: reboot/crash: '[STORE] saved\r\n[STATE] SLEEP timer=1 s\nets Jul 29 2019 12:21:46\r\n\r\nrst:0x5 (DEEPSLEEP_RESET),boot:0x13 (SPI_FAST_FLASH_BOOT)\r\nconfigsip: 0, SPIWP:0xee\r\nclk_drv:0x00,q_drv:0x00,d_drv:0x00,cs0_drv:0x00,hd_drv:0x00,wp_drv:0x00\r\nmode:DIO, clock div:2\r\nload:0x3fff0030,len:1156\r\nload:0x40078000,len:11456\r\nho 0 tail 12 room 4\r\nload:0x40080400,len:2972\r\nentry 0x400805dc\r\n[BOOT] reset_reason=DEEP_SLEEP\n[WAKE] cause=4\n[WDT] enabled timeout=4000 ms\nCommands:\r\n help\r\n status\r\n sensor\r\n relay on\r\n relay off\r\n save\r\n sleep <seconds>\r\n hang\r\n powerloss\r\n reboot\r\n'
E   assert not <re.Match object; span=(67, 71), match='rst:'>
E   + where <re.Match object; span=(67, 71), match='rst:'> = <function search at 0x7fa36afc9f70>('reset[_]reason|rst:|guru|panic|abort', '[STORE] saved\r\n[STATE] SLEEP timer=1 s\nets Jul 29 2019

12:21:46\r\n\r\nrst:0x5 (DEEPSLEEP_RESET),boot:0x13 (SPI_FAST_FLASH_BOOT)\r\nconfigsip: 0, SPIWP:0xee\r\nclk_drv:0x00,q_drv:0x00,d_drv:0x00,cs0_drv:0x00,hd_drv:0x00,wp_drv:0x00\r\nmode:DIO, clock div:2\r\nload:0x3fff0030,len:1156\r\nload:0x40078000,len:11456\r\nho 0 tail 12 room 4\r\nload:0x40080400,len:2972\r\nentry 0x400805dc\r\n[BOOT] reset_reason=DEEP_SLEEP\n[WAKE] cause=4\n[WDT] enabled timeout=4000 ms\nCommands:\r\n help\r\n status\r\n sensor\r\n relay on\r\n relay off\r\n save\r\n sleep <seconds>\r\n hang\r\n powerloss\r\n reboot\r\n', re.IGNORECASE)
E   + where <function search at 0x7fa36afc9f70> = re.search
E   + and   re.IGNORECASE = re.I

test/test_resilience.py:187: AssertionError
_____ test_invalid_sleep_args_after_recovery[abc-hang] _____

dut = <conftest.Dut object at 0x7fa36f736ee0>, fault_kind = 'hang', arg = 'abc'

```
    @pytest.mark.parametrize("fault_kind", ["hang", "powerloss", "sleep"])
    @pytest.mark.parametrize("arg", INVALID_SLEEP)              # TC-NEG-02..06, TC-PAR-03
    def test_invalid_sleep_args_after_recovery(dut, fault_kind, arg):
        fault(dut, fault_kind)
        dut.send(("sleep " + arg).rstrip())
        out = dut.read_for(3)
>       assert_alive_no_reboot(dut, out)
```

test/test_resilience.py:201:
_ _

dut = <conftest.Dut object at 0x7fa36f736ee0>
output = '[STORE] saved\r\n[STATE] SLEEP timer=1 s\nets Jul 29 2019 12:21:46\r\n\r\nrst:0x5 (DEEPSLEEP_RESET),boot:0x13 (SPI_FA...tatus\r\n sensor\r\n relay on\r\n relay off\r\n save\r\n sleep <seconds>\r\n hang\r\n powerloss\r\n reboot\r\n'

```
    def assert_alive_no_reboot(dut, output):
>       assert not re.search(r"reset[_ ]reason|rst:|guru|panic|abort", output, re.I), f"reboot/crash: {output!r}"
```
E   AssertionError: reboot/crash: '[STORE] saved\r\n[STATE] SLEEP timer=1 s\nets Jul 29 2019 12:21:46\r\n\r\nrst:0x5 (DEEPSLEEP_RESET),boot:0x13 (SPI_FAST_FLASH_BOOT)\r\nconfigsip: 0, SPIWP:0xee\r\nclk_drv:0x00,q_drv:0x00,d_drv:0x00,cs0_drv:0x00,hd_drv:0x00,wp_drv:0x00\r\nmode:DIO, clock div:2\r\nload:0x3fff0030,len:1156\r\nload:0x40078000,len:11456\r\nho 0 tail 12 room 4\r\nload:0x40080400,len:2972\r\nentry

0x400805dc\r\n[BOOT] reset_reason=DEEP_SLEEP\n[WAKE] cause=4\n[WDT] enabled timeout=4000 ms\nCommands:\r\n help\r\n status\r\n sensor\r\n relay on\r\n relay off\r\n save\r\n sleep <seconds>\r\n hang\r\n powerloss\r\n reboot\r\n'
E   assert not <re.Match object; span=(67, 71), match='rst:'>
E   + where <re.Match object; span=(67, 71), match='rst:'> = <function search at 0x7fa36afc9f70>('reset[_]reason|rst:|guru|panic|abort', '[STORE] saved\r\n[STATE] SLEEP timer=1 s\nets Jul 29 2019 12:21:46\r\n\r\nrst:0x5 (DEEPSLEEP_RESET),boot:0x13 (SPI_FAST_FLASH_BOOT)\r\nconfigsip: 0, SPIWP:0xee\r\nclk_drv:0x00,q_drv:0x00,d_drv:0x00,cs0_drv:0x00,hd_drv:0x00,wp_drv:0x00\r\nmode:DIO, clock div:2\r\nload:0x3fff0030,len:1156\r\nload:0x40078000,len:11456\r\nho 0 tail 12 room 4\r\nload:0x40080400,len:2972\r\nentry 0x400805dc\r\n[BOOT] reset_reason=DEEP_SLEEP\n[WAKE] cause=4\n[WDT] enabled timeout=4000 ms\nCommands:\r\n help\r\n status\r\n sensor\r\n relay on\r\n relay off\r\n save\r\n sleep <seconds>\r\n hang\r\n powerloss\r\n reboot\r\n', re.IGNORECASE)
E   +   where <function search at 0x7fa36afc9f70> = re.search
E   +   and   re.IGNORECASE = re.I

test/test_resilience.py:187: AssertionError
_____ test_invalid_sleep_args_after_recovery[abc-powerloss] _____

dut = <conftest.Dut object at 0x7fa36f736ee0>, fault_kind = 'powerloss'
arg = 'abc'

```
    @pytest.mark.parametrize("fault_kind", ["hang", "powerloss", "sleep"])
    @pytest.mark.parametrize("arg", INVALID_SLEEP)              # TC-NEG-02..06, TC-PAR-03
    def test_invalid_sleep_args_after_recovery(dut, fault_kind, arg):
        fault(dut, fault_kind)
        dut.send(("sleep " + arg).rstrip())
        out = dut.read_for(3)
>       assert_alive_no_reboot(dut, out)
```

test/test_resilience.py:201:
_ _

dut = <conftest.Dut object at 0x7fa36f736ee0>
output = '[BOOT] reset_reason=SOFTWARE_RESET\n[WDT] enabled timeout=4000 ms\nCommands:\r\n help\r\n status\r\n sensor\r\n r...tatus\r\n sensor\r\n relay on\r\n relay off\r\n save\r\n sleep <seconds>\r\n hang\r\n powerloss\r\n reboot\r\n'

```
    def assert_alive_no_reboot(dut, output):
```

> assert not re.search(r"reset[_]reason|rst:|guru|panic|abort", output, re.I), f"reboot/crash: {output!r}"
E AssertionError: reboot/crash: '[BOOT] reset_reason=SOFTWARE_RESET\n[WDT] enabled timeout=4000 ms\nCommands:\r\n help\r\n status\r\n sensor\r\n relay on\r\n relay off\r\n save\r\n sleep <seconds>\r\n hang\r\n powerloss\r\n reboot\r\n[STORE] saved\r\n[STATE] SLEEP timer=1 s\nets Jul 29 2019 12:21:46\r\n\r\nrst:0x5 (DEEPSLEEP_RESET),boot:0x13 (SPI_FAST_FLASH_BOOT)\r\nconfigsip: 0, SPIWP:0xee\r\nclk_drv:0x00,q_drv:0x00,d_drv:0x00,cs0_drv:0x00,hd_drv:0x00,wp_drv:0x00\r\nmode:DIO, clock div:2\r\nload:0x3fff0030,len:1156\r\nload:0x40078000,len:11456\r\nho 0 tail 12 room 4\r\nload:0x40080400,len:2972\r\nentry 0x400805dc\r\n[BOOT] reset_reason=DEEP_SLEEP\n[WAKE] cause=4\n[WDT] enabled timeout=4000 ms\nCommands:\r\n help\r\n status\r\n sensor\r\n relay on\r\n relay off\r\n save\r\n sleep <seconds>\r\n hang\r\n powerloss\r\n reboot\r\n'
E assert not <re.Match object; span=(7, 19), match='reset_reason'>
E + where <re.Match object; span=(7, 19), match='reset_reason'> = <function search at 0x7fa36afc9f70>('reset[_]reason|rst:|guru|panic|abort', '[BOOT] reset_reason=SOFTWARE_RESET\n[WDT] enabled timeout=4000 ms\nCommands:\r\n help\r\n status\r\n sensor\r\n relay on\r\n relay off\r\n save\r\n sleep <seconds>\r\n hang\r\n powerloss\r\n reboot\r\n[STORE] saved\r\n[STATE] SLEEP timer=1 s\nets Jul 29 2019 12:21:46\r\n\r\nrst:0x5 (DEEPSLEEP_RESET),boot:0x13 (SPI_FAST_FLASH_BOOT)\r\nconfigsip: 0, SPIWP:0xee\r\nclk_drv:0x00,q_drv:0x00,d_drv:0x00,cs0_drv:0x00,hd_drv:0x00,wp_drv:0x00\r\nmode:DIO, clock div:2\r\nload:0x3fff0030,len:1156\r\nload:0x40078000,len:11456\r\nho 0 tail 12 room 4\r\nload:0x40080400,len:2972\r\nentry 0x400805dc\r\n[BOOT] reset_reason=DEEP_SLEEP\n[WAKE] cause=4\n[WDT] enabled timeout=4000 ms\nCommands:\r\n help\r\n status\r\n sensor\r\n relay on\r\n relay off\r\n save\r\n sleep <seconds>\r\n hang\r\n powerloss\r\n reboot\r\n', re.IGNORECASE)
E + where <function search at 0x7fa36afc9f70> = re.search
E + and re.IGNORECASE = re.I

test/test_resilience.py:187: AssertionError
_____ test_invalid_sleep_args_after_recovery[abc-sleep] _____

dut = <conftest.Dut object at 0x7fa36f736ee0>, fault_kind = 'sleep', arg = 'abc'

```
    @pytest.mark.parametrize("fault_kind", ["hang", "powerloss", "sleep"])
    @pytest.mark.parametrize("arg", INVALID_SLEEP)              # TC-NEG-02..06, TC-PAR-03
    def test_invalid_sleep_args_after_recovery(dut, fault_kind, arg):
```

```
        fault(dut, fault_kind)
        dut.send(("sleep " + arg).rstrip())
        out = dut.read_for(3)
>       assert_alive_no_reboot(dut, out)
```

test/test_resilience.py:201:

_ _

```
dut = <conftest.Dut object at 0x7fa36f736ee0>
output = '[STORE] saved\r\n[STATE] SLEEP timer=1 s\nets Jul 29 2019
12:21:46\r\n\r\nrst:0x5 (DEEPSLEEP_RESET),boot:0x13 (SPI_FA...tatus\r\n
sensor\r\n  relay on\r\n  relay off\r\n  save\r\n  sleep <seconds>\r\n  hang\r\n
powerloss\r\n  reboot\r\n'

    def assert_alive_no_reboot(dut, output):
>       assert not re.search(r"reset[_ ]reason|rst:|guru|panic|abort", output, re.I),
f"reboot/crash: {output!r}"
E       AssertionError: reboot/crash: '[STORE] saved\r\n[STATE] SLEEP
timer=1 s\nets Jul 29 2019 12:21:46\r\n\r\nrst:0x5
(DEEPSLEEP_RESET),boot:0x13 (SPI_FAST_FLASH_BOOT)\r\nconfigsip: 0,
SPIWP:0xee\r\nclk_drv:0x00,q_drv:0x00,d_drv:0x00,cs0_drv:0x00,hd_dr
v:0x00,wp_drv:0x00\r\nmode:DIO, clock
div:2\r\nload:0x3fff0030,len:1156\r\nload:0x40078000,len:11456\r\nho 0
tail 12 room 4\r\nload:0x40080400,len:2972\r\nentry
0x400805dc\r\n[BOOT] reset_reason=DEEP_SLEEP\n[WAKE]
cause=4\n[WDT] enabled timeout=4000 ms\nCommands:\r\n  help\r\n
status\r\n  sensor\r\n  relay on\r\n  relay off\r\n  save\r\n  sleep
<seconds>\r\n  hang\r\n  powerloss\r\n  reboot\r\n'
E       assert not <re.Match object; span=(67, 71), match='rst:'>
E        +  where <re.Match object; span=(67, 71), match='rst:'> = <function
search at 0x7fa36afc9f70>('reset[_ ]reason|rst:|guru|panic|abort',
'[STORE] saved\r\n[STATE] SLEEP timer=1 s\nets Jul 29 2019
12:21:46\r\n\r\nrst:0x5 (DEEPSLEEP_RESET),boot:0x13
(SPI_FAST_FLASH_BOOT)\r\nconfigsip: 0,
SPIWP:0xee\r\nclk_drv:0x00,q_drv:0x00,d_drv:0x00,cs0_drv:0x00,hd_dr
v:0x00,wp_drv:0x00\r\nmode:DIO, clock
div:2\r\nload:0x3fff0030,len:1156\r\nload:0x40078000,len:11456\r\nho 0
tail 12 room 4\r\nload:0x40080400,len:2972\r\nentry
0x400805dc\r\n[BOOT] reset_reason=DEEP_SLEEP\n[WAKE]
cause=4\n[WDT] enabled timeout=4000 ms\nCommands:\r\n  help\r\n
status\r\n  sensor\r\n  relay on\r\n  relay off\r\n  save\r\n  sleep
<seconds>\r\n  hang\r\n  powerloss\r\n  reboot\r\n', re.IGNORECASE)
E       +    where <function search at 0x7fa36afc9f70> = re.search
E       +    and   re.IGNORECASE = re.I
```

test/test_resilience.py:187: AssertionError

_____ test_invalid_sleep_args_after_recovery[99999999999-

hang] _____

dut = <conftest.Dut object at 0x7fa36f736ee0>, fault_kind = 'hang'
arg = '99999999999'

```
    @pytest.mark.parametrize("fault_kind", ["hang", "powerloss", "sleep"])
    @pytest.mark.parametrize("arg", INVALID_SLEEP)              # TC-NEG-02..06,
TC-PAR-03
    def test_invalid_sleep_args_after_recovery(dut, fault_kind, arg):
        fault(dut, fault_kind)
        dut.send(("sleep " + arg).rstrip())
        out = dut.read_for(3)
>       assert_alive_no_reboot(dut, out)
```

test/test_resilience.py:201:

_ _

dut = <conftest.Dut object at 0x7fa36f736ee0>
output = '[STORE] saved\r\n[STATE] SLEEP timer=60 s\nets Jul 29 2019
12:21:46\r\n\r\nrst:0x8 (TG1WDT_SYS_RESET),boot:0x13 (SPI_...tatus\r\n
sensor\r\n relay on\r\n relay off\r\n save\r\n sleep <seconds>\r\n hang\r\n
powerloss\r\n reboot\r\n'

```
    def assert_alive_no_reboot(dut, output):
>       assert not re.search(r"reset[_ ]reason|rst:|guru|panic|abort", output, re.I),
f"reboot/crash: {output!r}"
```

**E   AssertionError: reboot/crash: '[STORE] saved\r\n[STATE] SLEEP timer=60 s\nets Jul 29 2019 12:21:46\r\n\r\nrst:0x8 (TG1WDT_SYS_RESET),boot:0x13 (SPI_FAST_FLASH_BOOT)\r\nconfigsip: 0,
SPIWP:0xee\r\nclk_drv:0x00,q_drv:0x00,d_drv:0x00,cs0_drv:0x00,hd_drv:0x00,wp_drv:0x00\r\nmode:DIO, clock div:2\r\nload:0x3fff0030,len:1156\r\nload:0x40078000,len:11456\r\nho 0 tail 12 room 4\r\nload:0x40080400,len:2972\r\nentry 0x400805dc\r\n[BOOT] reset_reason=INT_WDT\n[WDT] enabled timeout=4000 ms\nCommands:\r\n help\r\n status\r\n sensor\r\n relay on\r\n relay off\r\n save\r\n sleep <seconds>\r\n hang\r\n powerloss\r\n reboot\r\n'**
E   assert not <re.Match object; span=(68, 72), match='rst:'>
**E   + where <re.Match object; span=(68, 72), match='rst:'> = <function search at 0x7fa36afc9f70>('reset[_]reason|rst:|guru|panic|abort', '[STORE] saved\r\n[STATE] SLEEP timer=60 s\nets Jul 29 2019 12:21:46\r\n\r\nrst:0x8 (TG1WDT_SYS_RESET),boot:0x13 (SPI_FAST_FLASH_BOOT)\r\nconfigsip: 0,
SPIWP:0xee\r\nclk_drv:0x00,q_drv:0x00,d_drv:0x00,cs0_drv:0x00,hd_drv:0x00,wp_drv:0x00\r\nmode:DIO, clock div:2\r\nload:0x3fff0030,len:1156\r\nload:0x40078000,len:11456\r\nho 0**

tail 12 room 4\r\nload:0x40080400,len:2972\r\nentry 0x400805dc\r\n[BOOT] reset_reason=INT_WDT\n[WDT] enabled timeout=4000 ms\nCommands:\r\n help\r\n status\r\n sensor\r\n relay on\r\n relay off\r\n save\r\n sleep <seconds>\r\n hang\r\n powerloss\r\n reboot\r\n', re.IGNORECASE)
E + where <function search at 0x7fa36afc9f70> = re.search
E + and re.IGNORECASE = re.I

test/test_resilience.py:187: AssertionError
_____ test_invalid_sleep_args_after_recovery[99999999999-powerloss] _____

dut = <conftest.Dut object at 0x7fa36f736ee0>, fault_kind = 'powerloss'
arg = '99999999999'

```
    @pytest.mark.parametrize("fault_kind", ["hang", "powerloss", "sleep"])
    @pytest.mark.parametrize("arg", INVALID_SLEEP)              # TC-NEG-02..06, TC-PAR-03
    def test_invalid_sleep_args_after_recovery(dut, fault_kind, arg):
        fault(dut, fault_kind)
        dut.send(("sleep " + arg).rstrip())
        out = dut.read_for(3)
>       assert_alive_no_reboot(dut, out)
```

test/test_resilience.py:201:
_ _

dut = <conftest.Dut object at 0x7fa36f736ee0>
output = ' status\r\n sensor\r\n relay on\r\n relay off\r\n save\r\n sleep <seconds>\r\n hang\r\n powerloss\r\n reboot\...tatus\r\n sensor\r\n relay on\r\n relay off\r\n save\r\n sleep <seconds>\r\n hang\r\n powerloss\r\n reboot\r\n'

```
    def assert_alive_no_reboot(dut, output):
>       assert not re.search(r"reset[_ ]reason|rst:|guru|panic|abort", output, re.I), f"reboot/crash: {output!r}"
```
E AssertionError: reboot/crash: ' status\r\n sensor\r\n relay on\r\n relay off\r\n save\r\n sleep <seconds>\r\n hang\r\n powerloss\r\n reboot\r\n[STORE] saved\r\n[STATE] SLEEP timer=60 s\nets Jul 29 2019 12:21:46\r\n\r\nrst:0x8 (TG1WDT_SYS_RESET),boot:0x13 (SPI_FAST_FLASH_BOOT)\r\nconfigsip: 0, SPIWP:0xee\r\nclk_drv:0x00,q_drv:0x00,d_drv:0x00,cs0_drv:0x00,hd_drv:0x00,wp_drv:0x00\r\nmode:DIO, clock div:2\r\nload:0x3fff0030,len:1156\r\nload:0x40078000,len:11456\r\nho 0 tail 12 room 4\r\nload:0x40080400,len:2972\r\nentry 0x400805dc\r\n[BOOT] reset_reason=INT_WDT\n[WDT] enabled timeout=4000 ms\nCommands:\r\n help\r\n status\r\n sensor\r\n relay

on\r\n relay off\r\n save\r\n sleep <seconds>\r\n hang\r\n powerloss\r\n reboot\r\n'
E assert not <re.Match object; span=(170, 174), match='rst:'>
E + where <re.Match object; span=(170, 174), match='rst:'> = <function search at 0x7fa36afc9f70>('reset[_]reason|rst:|guru|panic|abort', ' status\r\n sensor\r\n relay on\r\n relay off\r\n save\r\n sleep <seconds>\r\n hang\r\n powerloss\r\n reboot\r\n[STORE] saved\r\n[STATE] SLEEP timer=60 s\nets Jul 29 2019 12:21:46\r\n\r\nrst:0x8 (TG1WDT_SYS_RESET),boot:0x13 (SPI_FAST_FLASH_BOOT)\r\nconfigsip: 0, SPIWP:0xee\r\nclk_drv:0x00,q_drv:0x00,d_drv:0x00,cs0_drv:0x00,hd_drv:0x00,wp_drv:0x00\r\nmode:DIO, clock div:2\r\nload:0x3fff0030,len:1156\r\nload:0x40078000,len:11456\r\nho 0 tail 12 room 4\r\nload:0x40080400,len:2972\r\nentry 0x400805dc\r\n[BOOT] reset_reason=INT_WDT\n[WDT] enabled timeout=4000 ms\nCommands:\r\n help\r\n status\r\n sensor\r\n relay on\r\n relay off\r\n save\r\n sleep <seconds>\r\n hang\r\n powerloss\r\n reboot\r\n', re.IGNORECASE)
E + where <function search at 0x7fa36afc9f70> = re.search
E + and re.IGNORECASE = re.I

test/test_resilience.py:187: AssertionError
_____ test_invalid_sleep_args_after_recovery[99999999999-sleep] _____

dut = <conftest.Dut object at 0x7fa36f736ee0>, fault_kind = 'sleep'
arg = '99999999999'

```
    @pytest.mark.parametrize("fault_kind", ["hang", "powerloss", "sleep"])
    @pytest.mark.parametrize("arg", INVALID_SLEEP)              # TC-NEG-02..06, TC-PAR-03
    def test_invalid_sleep_args_after_recovery(dut, fault_kind, arg):
        fault(dut, fault_kind)
        dut.send(("sleep " + arg).rstrip())
        out = dut.read_for(3)
>       assert_alive_no_reboot(dut, out)
```

test/test_resilience.py:201:
_ _

dut = <conftest.Dut object at 0x7fa36f736ee0>
output = '[STORE] saved\r\n[STATE] SLEEP timer=60 s\nets Jul 29 2019 12:21:46\r\n\r\nrst:0x8 (TG1WDT_SYS_RESET),boot:0x13 (SPI_...tatus\r\n sensor\r\n relay on\r\n relay off\r\n save\r\n sleep <seconds>\r\n hang\r\n powerloss\r\n reboot\r\n'

```
    def assert_alive_no_reboot(dut, output):
```

> assert not re.search(r"reset[_]reason|rst:|guru|panic|abort", output, re.I), f"reboot/crash: {output!r}"
E AssertionError: reboot/crash: '[STORE] saved\r\n[STATE] SLEEP timer=60 s\nets Jul 29 2019 12:21:46\r\n\r\nrst:0x8 (TG1WDT_SYS_RESET),boot:0x13 (SPI_FAST_FLASH_BOOT)\r\nconfigsip: 0, SPIWP:0xee\r\nclk_drv:0x00,q_drv:0x00,d_drv:0x00,cs0_drv:0x00,hd_drv:0x00,wp_drv:0x00\r\nmode:DIO, clock div:2\r\nload:0x3fff0030,len:1156\r\nload:0x40078000,len:11456\r\nho 0 tail 12 room 4\r\nload:0x40080400,len:2972\r\nentry 0x400805dc\r\n[BOOT] reset_reason=INT_WDT\n[WDT] enabled timeout=4000 ms\nCommands:\r\n help\r\n status\r\n sensor\r\n relay on\r\n relay off\r\n save\r\n sleep <seconds>\r\n hang\r\n powerloss\r\n reboot\r\n'
E assert not <re.Match object; span=(68, 72), match='rst:'>
E + where <re.Match object; span=(68, 72), match='rst:'> = <function search at 0x7fa36afc9f70>('reset[_]reason|rst:|guru|panic|abort', '[STORE] saved\r\n[STATE] SLEEP timer=60 s\nets Jul 29 2019 12:21:46\r\n\r\nrst:0x8 (TG1WDT_SYS_RESET),boot:0x13 (SPI_FAST_FLASH_BOOT)\r\nconfigsip: 0, SPIWP:0xee\r\nclk_drv:0x00,q_drv:0x00,d_drv:0x00,cs0_drv:0x00,hd_drv:0x00,wp_drv:0x00\r\nmode:DIO, clock div:2\r\nload:0x3fff0030,len:1156\r\nload:0x40078000,len:11456\r\nho 0 tail 12 room 4\r\nload:0x40080400,len:2972\r\nentry 0x400805dc\r\n[BOOT] reset_reason=INT_WDT\n[WDT] enabled timeout=4000 ms\nCommands:\r\n help\r\n status\r\n sensor\r\n relay on\r\n relay off\r\n save\r\n sleep <seconds>\r\n hang\r\n powerloss\r\n reboot\r\n', re.IGNORECASE)
E + where <function search at 0x7fa36afc9f70> = re.search
E + and re.IGNORECASE = re.I

test/test_resilience.py:187: AssertionError
_____ test_invalid_sleep_args_after_recovery[3.5.1-hang] _____

dut = <conftest.Dut object at 0x7fa36f736ee0>, fault_kind = 'hang'
arg = '3.5.1'

```
    @pytest.mark.parametrize("fault_kind", ["hang", "powerloss", "sleep"])
    @pytest.mark.parametrize("arg", INVALID_SLEEP)              # TC-NEG-02..06, TC-PAR-03
    def test_invalid_sleep_args_after_recovery(dut, fault_kind, arg):
        fault(dut, fault_kind)
        dut.send(("sleep " + arg).rstrip())
        out = dut.read_for(3)
>       assert_alive_no_reboot(dut, out)
```

test/test_resilience.py:201:
_ _

dut = <conftest.Dut object at 0x7fa36f736ee0>
output = '[STORE] saved\r\n[STATE] SLEEP timer=3 s\nets Jul 29 2019 12:21:46\r\n\r\nrst:0x5 (DEEPSLEEP_RESET),boot:0x13 (SPI_FA...tatus\r\n sensor\r\n relay on\r\n relay off\r\n save\r\n sleep <seconds>\r\n hang\r\n powerloss\r\n reboot\r\n'

    def assert_alive_no_reboot(dut, output):
>   assert not re.search(r"reset[_]reason|rst:|guru|panic|abort", output, re.I), f"reboot/crash: {output!r}"
E   AssertionError: reboot/crash: '[STORE] saved\r\n[STATE] SLEEP timer=3 s\nets Jul 29 2019 12:21:46\r\n\r\nrst:0x5 (DEEPSLEEP_RESET),boot:0x13 (SPI_FAST_FLASH_BOOT)\r\nconfigsip: 0, SPIWP:0xee\r\nclk_drv:0x00,q_drv:0x00,d_drv:0x00,cs0_drv:0x00,hd_drv:0x00,wp_drv:0x00\r\nmode:DIO, clock div:2\r\nload:0x3fff0030,len:1156\r\nload:0x40078000,len:11456\r\nho 0 tail 12 room 4\r\nload:0x40080400,len:2972\r\nentry 0x400805dc\r\n[BOOT] reset_reason=DEEP_SLEEP\n[WAKE] cause=4\n[WDT] enabled timeout=4000 ms\nCommands:\r\n help\r\n status\r\n sensor\r\n relay on\r\n relay off\r\n save\r\n sleep <seconds>\r\n hang\r\n powerloss\r\n reboot\r\n'
E   assert not <re.Match object; span=(67, 71), match='rst:'>
E   + where <re.Match object; span=(67, 71), match='rst:'> = <function search at 0x7fa36afc9f70>('reset[_]reason|rst:|guru|panic|abort', '[STORE] saved\r\n[STATE] SLEEP timer=3 s\nets Jul 29 2019 12:21:46\r\n\r\nrst:0x5 (DEEPSLEEP_RESET),boot:0x13 (SPI_FAST_FLASH_BOOT)\r\nconfigsip: 0, SPIWP:0xee\r\nclk_drv:0x00,q_drv:0x00,d_drv:0x00,cs0_drv:0x00,hd_drv:0x00,wp_drv:0x00\r\nmode:DIO, clock div:2\r\nload:0x3fff0030,len:1156\r\nload:0x40078000,len:11456\r\nho 0 tail 12 room 4\r\nload:0x40080400,len:2972\r\nentry 0x400805dc\r\n[BOOT] reset_reason=DEEP_SLEEP\n[WAKE] cause=4\n[WDT] enabled timeout=4000 ms\nCommands:\r\n help\r\n status\r\n sensor\r\n relay on\r\n relay off\r\n save\r\n sleep <seconds>\r\n hang\r\n powerloss\r\n reboot\r\n', re.IGNORECASE)
E   +   where <function search at 0x7fa36afc9f70> = re.search
E   +   and   re.IGNORECASE = re.I

test/test_resilience.py:187: AssertionError
_____ test_invalid_sleep_args_after_recovery[3.5.1-powerloss] _____

dut = <conftest.Dut object at 0x7fa36f736ee0>, fault_kind = 'powerloss'
arg = '3.5.1'

```
    @pytest.mark.parametrize("fault_kind", ["hang", "powerloss", "sleep"])
    @pytest.mark.parametrize("arg", INVALID_SLEEP)              # TC-NEG-02..06,
TC-PAR-03
    def test_invalid_sleep_args_after_recovery(dut, fault_kind, arg):
        fault(dut, fault_kind)
        dut.send(("sleep " + arg).rstrip())
        out = dut.read_for(3)
>       assert_alive_no_reboot(dut, out)
```

test/test_resilience.py:201:

```
_ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _
```

```
dut = <conftest.Dut object at 0x7fa36f736ee0>
output = '[BOOT] reset_reason=SOFTWARE_RESET\n[WDT] enabled
timeout=4000 ms\nCommands:\r\n  help\r\n  status\r\n  sensor\r\n  r...tatus\r\n
sensor\r\n  relay on\r\n  relay off\r\n  save\r\n  sleep <seconds>\r\n  hang\r\n
powerloss\r\n  reboot\r\n'

    def assert_alive_no_reboot(dut, output):
>       assert not re.search(r"reset[_ ]reason|rst:|guru|panic|abort", output, re.I),
f"reboot/crash: {output!r}"
E       AssertionError: reboot/crash: '[BOOT]
reset_reason=SOFTWARE_RESET\n[WDT] enabled timeout=4000
ms\nCommands:\r\n  help\r\n  status\r\n  sensor\r\n  relay on\r\n  relay
off\r\n  save\r\n  sleep <seconds>\r\n  hang\r\n  powerloss\r\n
reboot\r\n[STORE] saved\r\n[STATE] SLEEP timer=3 s\nets Jul 29 2019
12:21:46\r\n\r\nrst:0x5 (DEEPSLEEP_RESET),boot:0x13
(SPI_FAST_FLASH_BOOT)\r\nconfigsip: 0,
SPIWP:0xee\r\nclk_drv:0x00,q_drv:0x00,d_drv:0x00,cs0_drv:0x00,hd_dr
v:0x00,wp_drv:0x00\r\nmode:DIO, clock
div:2\r\nload:0x3fff0030,len:1156\r\nload:0x40078000,len:11456\r\nho 0
tail 12 room 4\r\nload:0x40080400,len:2972\r\nentry
0x400805dc\r\n[BOOT] reset_reason=DEEP_SLEEP\n[WAKE]
cause=4\n[WDT] enabled timeout=4000 ms\nCommands:\r\n  help\r\n
status\r\n  sensor\r\n  relay on\r\n  relay off\r\n  save\r\n  sleep
<seconds>\r\n  hang\r\n  powerloss\r\n  reboot\r\n'
E       assert not <re.Match object; span=(7, 19), match='reset_reason'>
E        +  where <re.Match object; span=(7, 19), match='reset_reason'> =
<function search at 0x7fa36afc9f70>('reset[_ ]reason|rst:|guru|panic|
abort', '[BOOT] reset_reason=SOFTWARE_RESET\n[WDT] enabled
timeout=4000 ms\nCommands:\r\n  help\r\n  status\r\n  sensor\r\n  relay
on\r\n  relay off\r\n  save\r\n  sleep <seconds>\r\n  hang\r\n  powerloss\r\n
reboot\r\n[STORE] saved\r\n[STATE] SLEEP timer=3 s\nets Jul 29 2019
12:21:46\r\n\r\nrst:0x5 (DEEPSLEEP_RESET),boot:0x13
(SPI_FAST_FLASH_BOOT)\r\nconfigsip: 0,
SPIWP:0xee\r\nclk_drv:0x00,q_drv:0x00,d_drv:0x00,cs0_drv:0x00,hd_dr
```

v:0x00,wp_drv:0x00\r\nmode:DIO, clock div:2\r\nload:0x3fff0030,len:1156\r\nload:0x40078000,len:11456\r\nho 0 tail 12 room 4\r\nload:0x40080400,len:2972\r\nentry 0x400805dc\r\n[BOOT] reset_reason=DEEP_SLEEP\n[WAKE] cause=4\n[WDT] enabled timeout=4000 ms\nCommands:\r\n help\r\n status\r\n sensor\r\n relay on\r\n relay off\r\n save\r\n sleep <seconds>\r\n hang\r\n powerloss\r\n reboot\r\n', re.IGNORECASE)
E   + where <function search at 0x7fa36afc9f70> = re.search
E   + and   re.IGNORECASE = re.I

test/test_resilience.py:187: AssertionError
_____ test_invalid_sleep_args_after_recovery[3.5.1-sleep] _____

dut = <conftest.Dut object at 0x7fa36f736ee0>, fault_kind = 'sleep'
arg = '3.5.1'

    @pytest.mark.parametrize("fault_kind", ["hang", "powerloss", "sleep"])
    @pytest.mark.parametrize("arg", INVALID_SLEEP)   # TC-NEG-02..06, TC-PAR-03
    def test_invalid_sleep_args_after_recovery(dut, fault_kind, arg):
        fault(dut, fault_kind)
        dut.send(("sleep " + arg).rstrip())
        out = dut.read_for(3)
>   assert_alive_no_reboot(dut, out)

test/test_resilience.py:201:
_ _

dut = <conftest.Dut object at 0x7fa36f736ee0>
output = '[STORE] saved\r\n[STATE] SLEEP timer=3 s\nets Jul 29 2019 12:21:46\r\n\r\nrst:0x5 (DEEPSLEEP_RESET),boot:0x13 (SPI_FA...tatus\r\n sensor\r\n relay on\r\n relay off\r\n save\r\n sleep <seconds>\r\n hang\r\n powerloss\r\n reboot\r\n'

    def assert_alive_no_reboot(dut, output):
>   assert not re.search(r"reset[_]reason|rst:|guru|panic|abort", output, re.I), f"reboot/crash: {output!r}"
E   AssertionError: reboot/crash: '[STORE] saved\r\n[STATE] SLEEP timer=3 s\nets Jul 29 2019 12:21:46\r\n\r\nrst:0x5 (DEEPSLEEP_RESET),boot:0x13 (SPI_FAST_FLASH_BOOT)\r\nconfigsip: 0, SPIWP:0xee\r\nclk_drv:0x00,q_drv:0x00,d_drv:0x00,cs0_drv:0x00,hd_drv:0x00,wp_drv:0x00\r\nmode:DIO, clock div:2\r\nload:0x3fff0030,len:1156\r\nload:0x40078000,len:11456\r\nho 0 tail 12 room 4\r\nload:0x40080400,len:2972\r\nentry 0x400805dc\r\n[BOOT] reset_reason=DEEP_SLEEP\n[WAKE] cause=4\n[WDT] enabled timeout=4000 ms\nCommands:\r\n help\r\n

status\r\n sensor\r\n relay on\r\n relay off\r\n save\r\n sleep <seconds>\r\n hang\r\n powerloss\r\n reboot\r\n'
E   assert not <re.Match object; span=(67, 71), match='rst:'>
E   + where <re.Match object; span=(67, 71), match='rst:'> = <function search at 0x7fa36afc9f70>('reset[_]reason|rst:|guru|panic|abort', '[STORE] saved\r\n[STATE] SLEEP timer=3 s\nets Jul 29 2019 12:21:46\r\n\r\nrst:0x5 (DEEPSLEEP_RESET),boot:0x13 (SPI_FAST_FLASH_BOOT)\r\nconfigsip: 0, SPIWP:0xee\r\nclk_drv:0x00,q_drv:0x00,d_drv:0x00,cs0_drv:0x00,hd_drv:0x00,wp_drv:0x00\r\nmode:DIO, clock div:2\r\nload:0x3fff0030,len:1156\r\nload:0x40078000,len:11456\r\nho 0 tail 12 room 4\r\nload:0x40080400,len:2972\r\nentry 0x400805dc\r\n[BOOT] reset_reason=DEEP_SLEEP\n[WAKE] cause=4\n[WDT] enabled timeout=4000 ms\nCommands:\r\n help\r\n status\r\n sensor\r\n relay on\r\n relay off\r\n save\r\n sleep <seconds>\r\n hang\r\n powerloss\r\n reboot\r\n', re.IGNORECASE)
E   +   where <function search at 0x7fa36afc9f70> = re.search
E   +   and   re.IGNORECASE = re.I

test/test_resilience.py:187: AssertionError
_____ test_command_flood _____

dut = <conftest.Dut object at 0x7fa36f736ee0>

```
    def test_command_flood(dut):                          # TC-NEG-12
        for _ in range(50):
            dut.send("status")
        out = dut.read_for(5)
>       assert not re.search(WDT_RE, out), "WDT reset during command flood"
```
E   AssertionError: WDT reset during command flood
E   assert not <re.Match object; span=(34, 41), match='INT_WDT'>
E   + where <re.Match object; span=(34, 41), match='INT_WDT'> = <function search at 0x7fa36afc9f70>('(TASK_WDT|INT_WDT|WDT|WATCHDOG)', '{"state":"ACTIVE","reset_reason":"INT_WDT","relay":false,"counter":833,"heap":336172,"sensor_ok":true}\r\n{"state":"ACTIVE","reset_reason":"INT_WDT","relay":false,"counter":833,"heap":336172,"sensor_ok":true}\r\n{"state":"ACTIVE","reset_reason":"INT_WDT","relay":false,"counter":833,"heap":336172,"sensor_ok":true}\r\n{"state":"ACTIVE","reset_reason":"INT_WDT","relay":false,"counter":833,"heap":336172,"sensor_ok":true}\r\n{"state":"ACTIVE","reset_reason":"INT_WDT","relay":false,"counter":833,"heap":336172,"sensor_ok":true}

\r\n{"state":"ACTIVE","reset_reason":"INT_WDT","relay":false,"counter":833,"heap":336172,"sensor_ok":true}
\r\n{"state":"ACTIVE","reset_reason":"INT_WDT","relay":false,"counter":833,"heap":336172,"sensor_ok":true}
\r\n{"state":"ACTIVE","reset_reason":"INT_WDT","relay":false,"counter":833,"heap":336172,"sensor_ok":true}
\r\n{"state":"ACTIVE","reset_reason":"INT_WDT","relay":false,"counter":833,"heap":336172,"sensor_ok":true}
\r\n{"state":"ACTIVE","reset_reason":"INT_WDT","relay":false,"counter":833,"heap":336172,"sensor_ok":true}
\r\n{"state":"ACTIVE","reset_reason...ap":336172,"sensor_ok":true}
\r\n{"state":"ACTIVE","reset_reason":"INT_WDT","relay":false,"counter":833,"heap":336172,"sensor_ok":true}
\r\n{"state":"ACTIVE","reset_reason":"INT_WDT","relay":false,"counter":833,"heap":336172,"sensor_ok":true}
\r\n{"state":"ACTIVE","reset_reason":"INT_WDT","relay":false,"counter":833,"heap":336172,"sensor_ok":true}
\r\n{"state":"ACTIVE","reset_reason":"INT_WDT","relay":false,"counter":833,"heap":336172,"sensor_ok":true}
\r\n{"state":"ACTIVE","reset_reason":"INT_WDT","relay":false,"counter":833,"heap":336172,"sensor_ok":true}
\r\n{"state":"ACTIVE","reset_reason":"INT_WDT","relay":false,"counter":833,"heap":336172,"sensor_ok":true}
\r\n{"state":"ACTIVE","reset_reason":"INT_WDT","relay":false,"counter":833,"heap":336172,"sensor_ok":true}
\r\n{"state":"ACTIVE","reset_reason":"INT_WDT","relay":false,"counter":833,"heap":336172,"sensor_ok":true}
\r\n{"state":"ACTIVE","reset_reason":"INT_WDT","relay":false,"counter":833,"heap":336172,"sensor_ok":true}
\r\n{"state":"ACTIVE","reset_reason":"INT_WDT","relay":false,"counter":833,"heap":336172,"sensor_ok":true}
\r\n{"state":"ACTIVE","reset_reason":"INT_WDT","relay":false,"counter":833,"heap":336172,"sensor_ok":true}\r\n')
E + where <function search at 0x7fa36afc9f70> = re.search

test/test_resilience.py:226: AssertionError
========================== short test summary info ==========================
FAILED test/test_resilience.py::**test_wake_time_within_tolerance[3]** - AssertionError: sleep 3s: woke after 0.41s, allowed 2.55-3.45
FAILED test/test_resilience.py::**test_wake_time_within_tolerance[5]** - AssertionError: sleep 5s: woke after 0.59s, allowed 4.25-5.75
FAILED test/test_resilience.py::**test_wake_time_within_tolerance[10]** - AssertionError: sleep 10s: woke after 0.70s, allowed 8.50-11.50
FAILED test/test_resilience.py::**test_wake_time_within_tolerance[20]** - AssertionError: sleep 20s: woke after 0.71s, allowed 17.00-23.00
FAILED test/test_resilience.py::**test_repeated_sleep_cycles_and_heap** -

assert (3 * (1 - 0.15)) <= 0.39009260499999954
FAILED test/test_resilience.py::**test_wdt_fires_in_time** - AssertionError: WDT reset after 14.59s
FAILED test/test_resilience.py::**test_event_counter_sane_after_recovery** - AssertionError: no status reply, got: '[BOOT] reset_reason=SOFTWARE_RESET\n...
FAILED test/test_resilience.py::**test_invalid_sleep_args_after_recovery[-hang]** - AssertionError: reboot/crash: '[STORE] saved\r\n[STATE] SLEEP timer=5 s\net...
FAILED test/test_resilience.py::**test_invalid_sleep_args_after_recovery[-powerloss]** - AssertionError: reboot/crash: '[STORE] saved\r\n[STATE] SLEEP timer=5 s\net...
FAILED test/test_resilience.py::**test_invalid_sleep_args_after_recovery[-sleep]** - AssertionError: reboot/crash: '[STORE] saved\r\n[STATE] SLEEP timer=5 s\net...
FAILED test/test_resilience.py::**test_invalid_sleep_args_after_recovery[-5-hang]** - AssertionError: reboot/crash: '[STORE] saved\r\n[STATE] SLEEP timer=60 s\ne...
FAILED test/test_resilience.py::**test_invalid_sleep_args_after_recovery[-5-powerloss]** - AssertionError: reboot/crash: 'reason=SOFTWARE_RESET\n[WDT] enabled timeout...
FAILED test/test_resilience.py::**test_invalid_sleep_args_after_recovery[-5-sleep]** - AssertionError: reboot/crash: '[STORE] saved\r\n[STATE] SLEEP timer=60 s\ne...
FAILED test/test_resilience.py::**test_invalid_sleep_args_after_recovery[0-hang]** - AssertionError: reboot/crash: '[STORE] saved\r\n[STATE] SLEEP timer=1 s\net...
FAILED test/test_resilience.py::**test_invalid_sleep_args_after_recovery[0-powerloss]** - AssertionError: reboot/crash: '[BOOT] reset_reason=SOFTWARE_RESET\n[WDT] en...
FAILED test/test_resilience.py::**test_invalid_sleep_args_after_recovery[0-sleep]** - AssertionError: reboot/crash: '[STORE] saved\r\n[STATE] SLEEP timer=1 s\net...
FAILED test/test_resilience.py::**test_invalid_sleep_args_after_recovery[abc-hang]** - AssertionError: reboot/crash: '[STORE] saved\r\n[STATE] SLEEP timer=1 s\net...
FAILED test/test_resilience.py::**test_invalid_sleep_args_after_recovery[abc-powerloss]** - AssertionError: reboot/crash: '[BOOT] reset_reason=SOFTWARE_RESET\n[WDT] en...
FAILED test/test_resilience.py::**test_invalid_sleep_args_after_recovery[abc-sleep]** - AssertionError: reboot/crash: '[STORE] saved\r\n[STATE] SLEEP timer=1 s\net...
FAILED test/test_resilience.py::**test_invalid_sleep_args_after_recovery[99999999999-hang]** - AssertionError: reboot/crash: '[STORE] saved\r\n[STATE] SLEEP timer=60 s\ne...
FAILED test/

test_resilience.py::**test_invalid_sleep_args_after_recovery[99999999999-powerloss]** - AssertionError: reboot/crash: ' status\r\n sensor\r\n relay on\r\n relay...
FAILED test/test_resilience.py::**test_invalid_sleep_args_after_recovery[99999999999-sleep]** - AssertionError: reboot/crash: '[STORE] saved\r\n[STATE] SLEEP timer=60 s\ne...
FAILED test/test_resilience.py::**test_invalid_sleep_args_after_recovery[3.5.1-hang]** - AssertionError: reboot/crash: '[STORE] saved\r\n[STATE] SLEEP timer=3 s\net...
FAILED test/test_resilience.py::**test_invalid_sleep_args_after_recovery[3.5.1-powerloss]** - AssertionError: reboot/crash: '[BOOT] reset_reason=SOFTWARE_RESET\n[WDT] en...
FAILED test/test_resilience.py::**test_invalid_sleep_args_after_recovery[3.5.1-sleep]** - AssertionError: reboot/crash: '[STORE] saved\r\n[STATE] SLEEP timer=3 s\net...
FAILED test/test_resilience.py::**test_command_flood** - AssertionError: WDT reset during command flood
ERROR test/test_resilience.py::**test_actuator_off_after_wdt** - AssertionError: no status reply, got: '[BOOT] reset_reason=INT_WDT\n[WDT] e...
======== **26 failed**, 18 passed, 4 skipped, **1 error** in 467.53s (0:07:47) =========
(.venv) (base) Sergio@SERHIIs-MacBook-Pro homework_buggy %